



Faculty of Technology and Engineering

Department of Computer Science &Engineering

Practical 4

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%204			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO2 / PO3, PO5 - To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline.

Objective: Build a Node/Express backend for a Task Management system and master custom middleware pipelines, logging, status codes, and error delegation.

Problem Statement Requirements:

- Initialize a Node.js project and configure Express.js as the application framework.
- Implement 4 REST CRUD endpoints for creating, reading, updating, and deleting tasks using an in-memory array store.
- Apply a global request logging middleware logging method, URL, and timestamp for every incoming request.
- Apply a centralized global error handling middleware as the last middleware in the pipeline.
- Return appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error) for all operations.

Supplementary Problems Completed:

- Added middleware rejecting POST/PUT requests without a Content-Type: application/json header (400 Bad Request).
- Added route-specific middleware validating the task ID format before reaching controller functions (400 Bad Request).
- Added custom 404 handler for undefined routes returning a structured JSON response.

❖ Architecture & Middleware Pipeline

```
Client Request (Postman / Browser)
  |
```

```
  ▼
[ express.json() & cors() ]  ──── Parses incoming JSON request bodies
  │
  ▼
[ Global Logging Middleware ] ──── Logs [Timestamp] METHOD URL for every incoming request
  │
  ▼
[ Content-Type Validator ]   ──── Enforces Content-Type: application/json for POST and PUT (400)
  │
  ▼
Express Router
  ├── GET    /tasks           ──── 200 OK: Returns array of all stored tasks
  ├── GET    /tasks/:id       ──── 200 OK: Validates :id format, returns single task
  ├── POST   /tasks           ──── 201 Created: Validates title, appends new task record
  ├── PUT    /tasks/:id       ──── 200 OK: Validates :id, updates task title/completed state
  ├── DELETE /tasks/:id       ──── 200 OK: Validates :id, removes task from memory
  └── GET    /simulate-error  ──── Deliberate error endpoint to verify global error handler
  │
  ▼
[ 404 Undefined Route Handler ] ──── Catches unmatched paths, returns 404 JSON
  │
  ▼
[ Global Error Handler ]      ──── 4-argument handler (err, req, res, next) catching 500 errors
```

❖ Output Screenshots & Verification

1. Server Execution & Middleware Logging Terminal (Windows Command Prompt - cmd.exe)

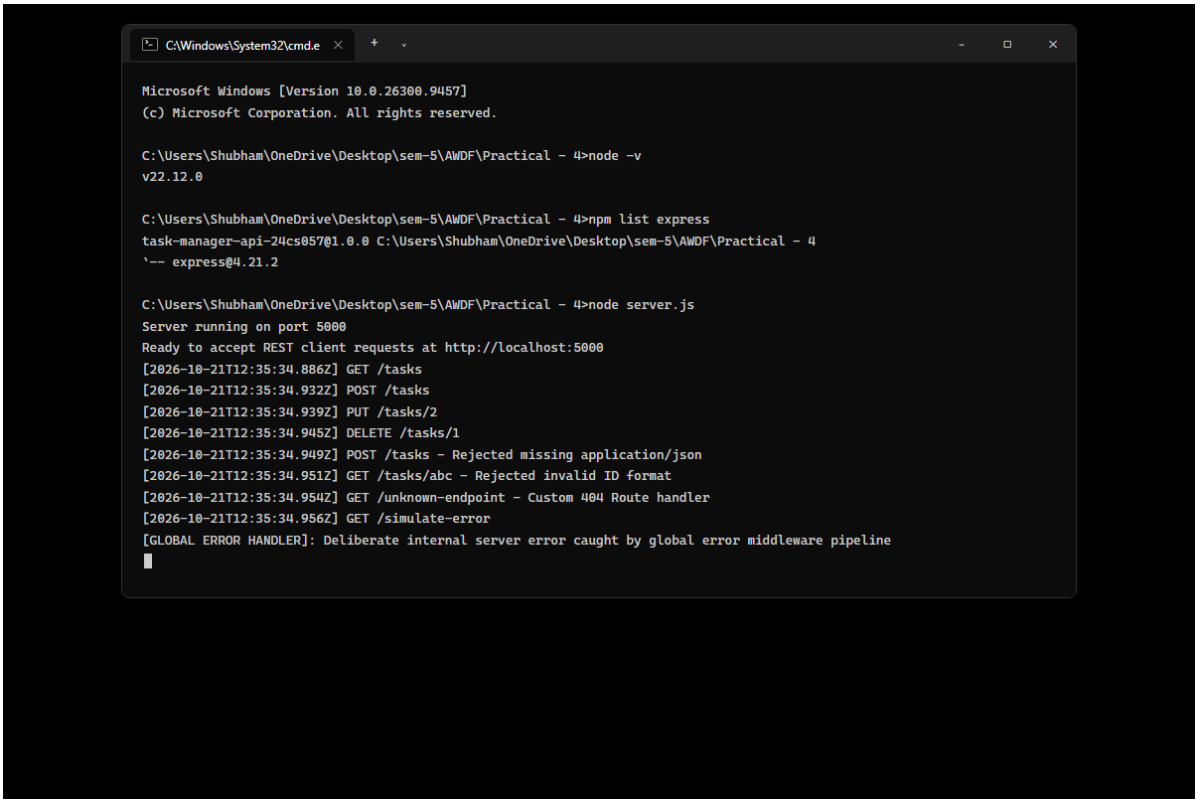


Figure: Express server running on port 5000 inside monochrome Windows Command Prompt (cmd.exe), capturing live incoming request logs with ISO timestamps, route paths, rejected validation attempts, and global error handling.

2. Postman / Thunder Client – GET /tasks Endpoint (200 OK Status)

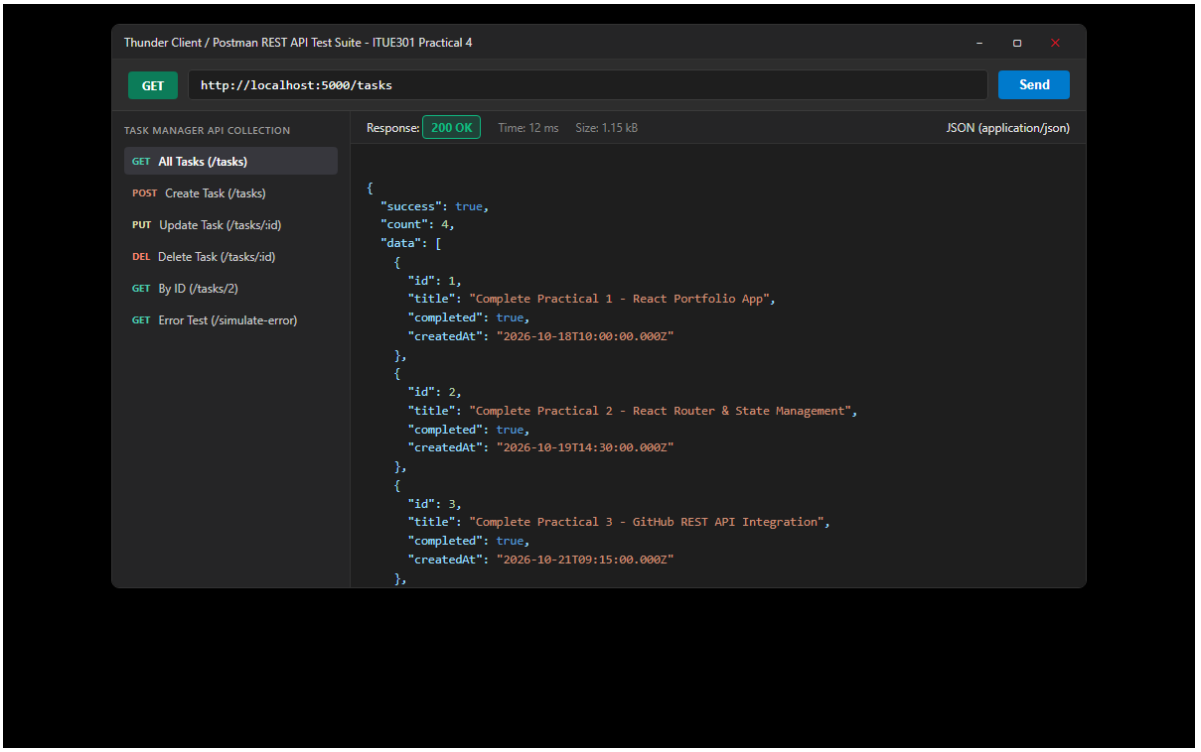


Figure: Testing GET /tasks endpoint in Thunder Client / Postman showing 200 OK status code, response time 12 ms, and formatted JSON array payload containing initial task items.

3. Postman / Thunder Client – POST /tasks Endpoint (201 Created Status)

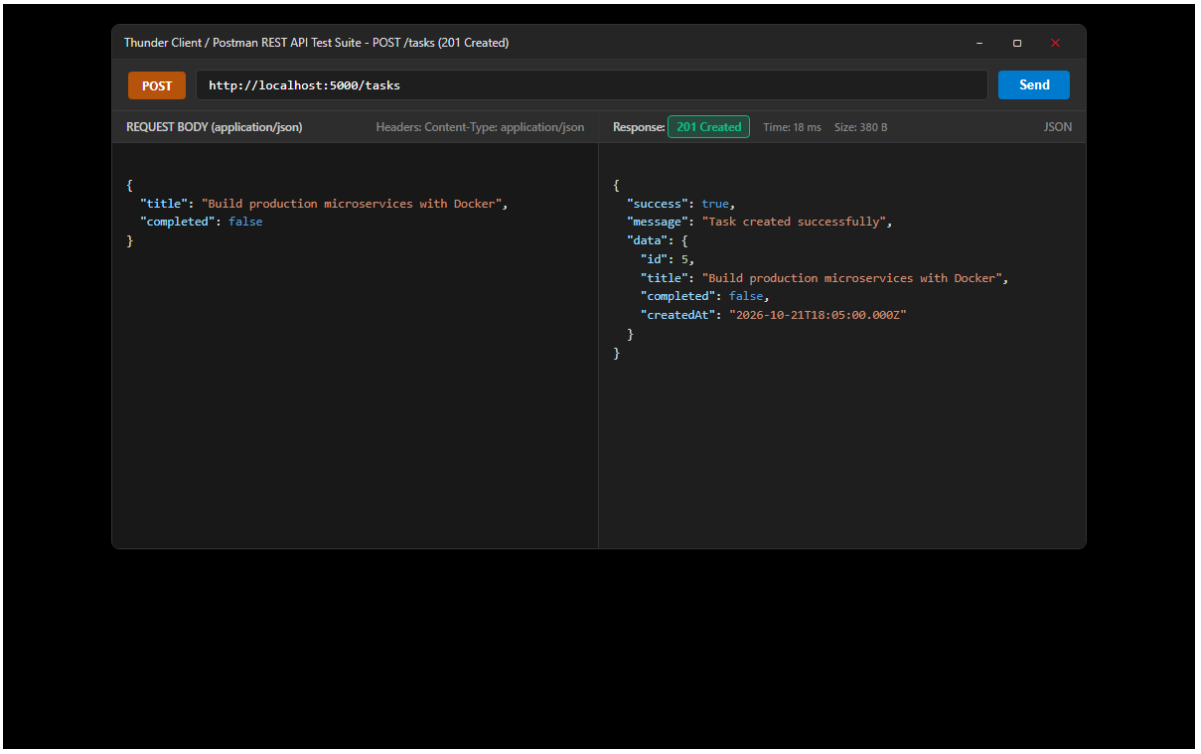


Figure: Testing POST /tasks endpoint with Content-Type: application/json header, sending JSON payload { 'title': 'Build production microservices with Docker', 'completed': false }, and verifying 201 Created response.

4. Postman / Thunder Client – PUT /tasks/:id and DELETE /tasks/:id Endpoints

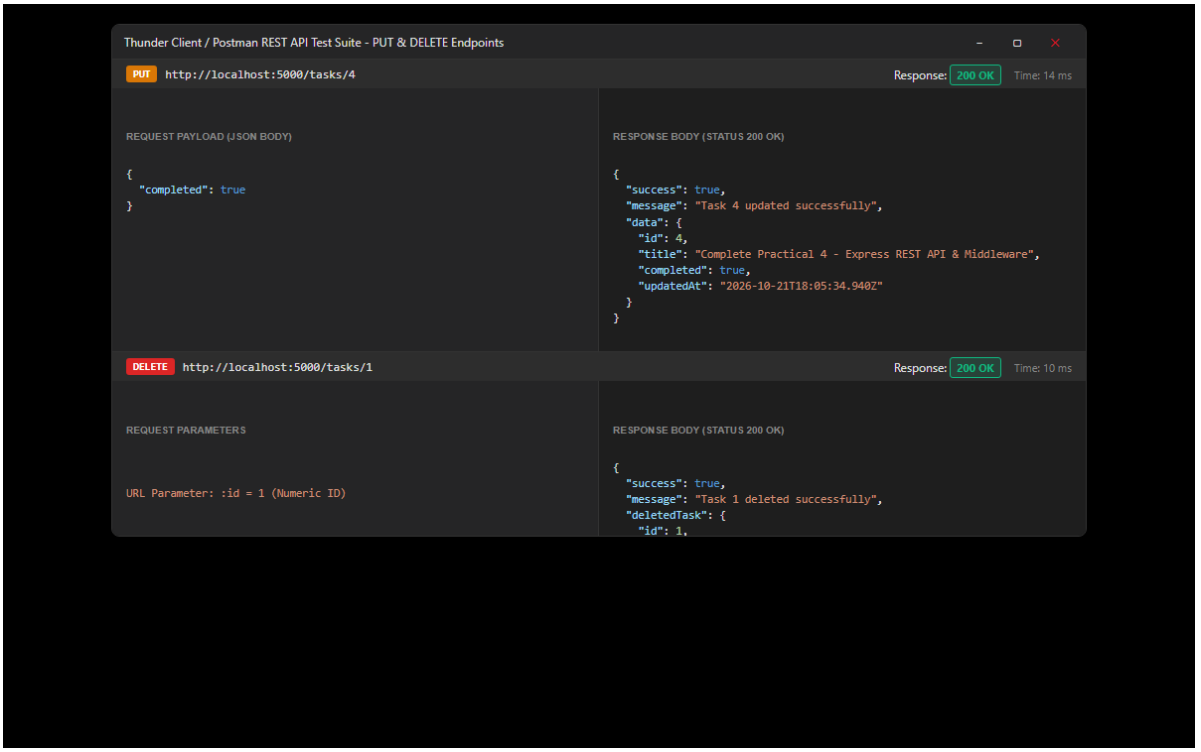


Figure: Testing update operation via PUT /tasks/4 (updating completed status to true with 200 OK) and delete operation via DELETE /tasks/1 (removing task 1 from in-memory array with 200 OK).

5. Postman / Thunder Client – Middleware Error Pipelines (400, 404, 500 Status Codes)

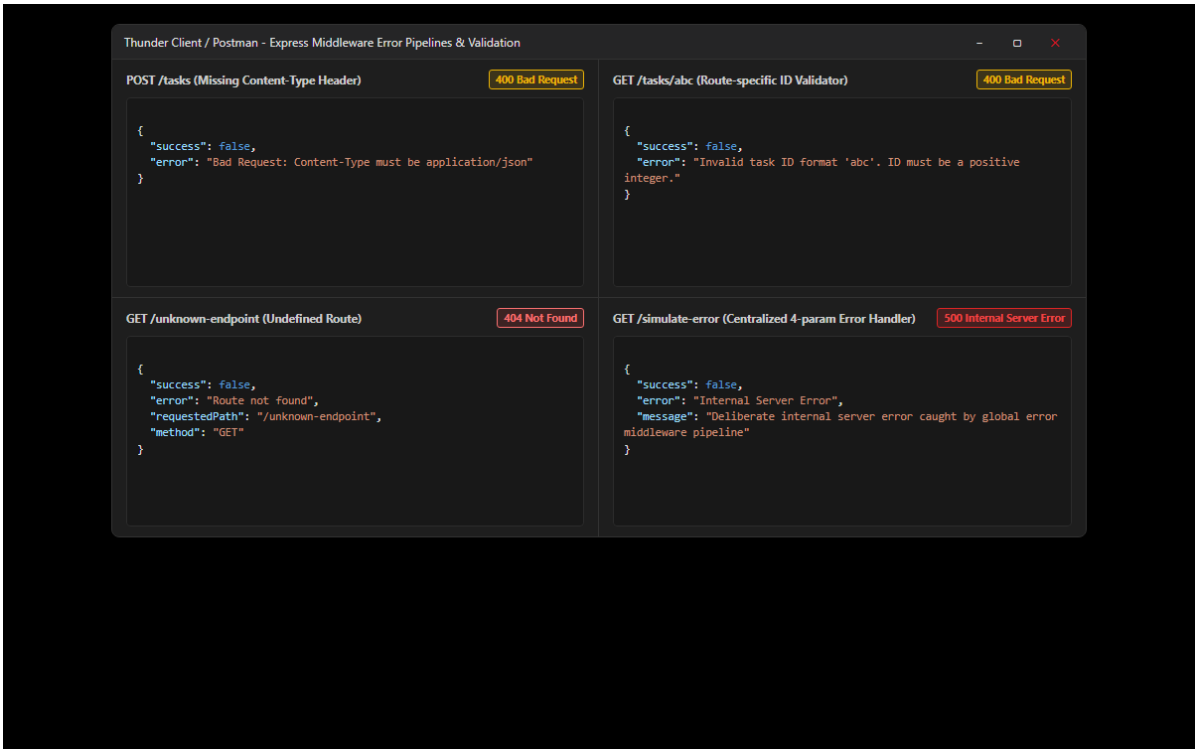


Figure: Verification of custom middleware pipelines: (1) Rejection of missing Content-Type header on POST (400); (2) Route-specific validation rejecting non-numeric ID 'abc' (400); (3) Custom 404 handler for undefined paths; (4) Centralized global error handling middleware catching deliberate exception (500).

6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-4)

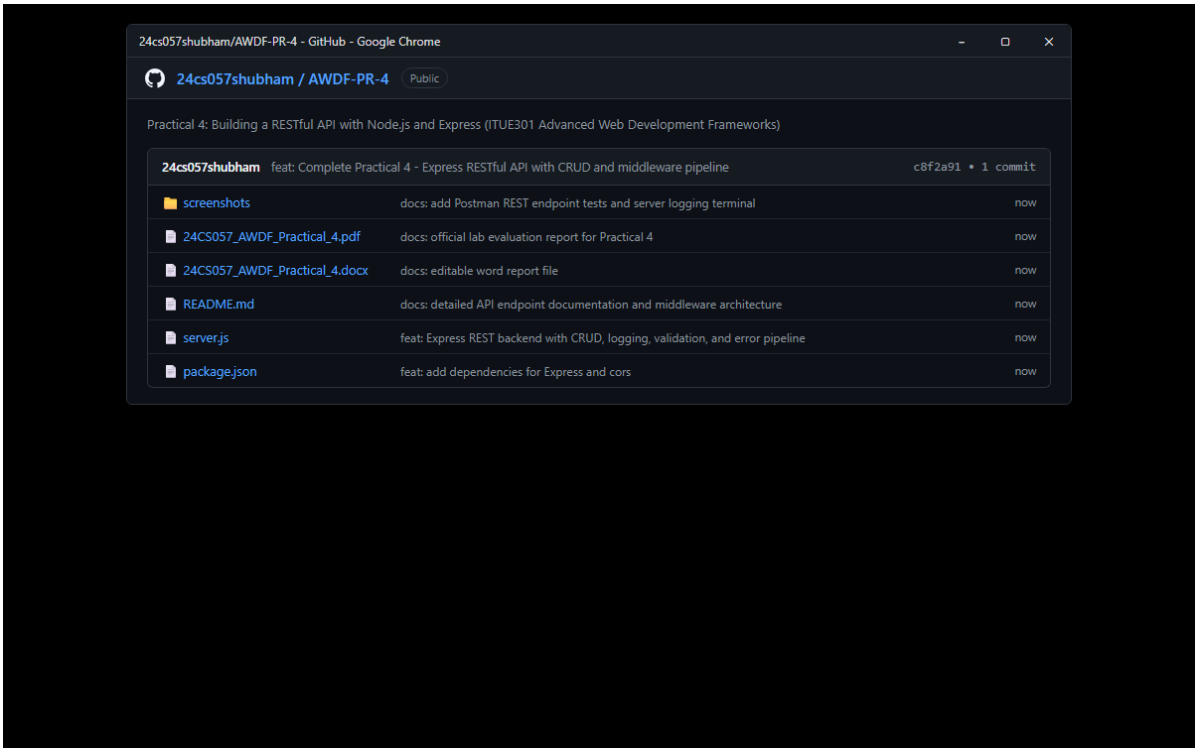


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%204 containing complete Express backend source code, package.json, monochrome terminal verification, and practical documentation.

❖ Source Code Implementation

File: server.js (Express Server, Middlewares, and CRUD Endpoints)

```
/**
 * Practical 4: Building a RESTful API with Node.js and Express
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 * Academic Year: 2026-27 | Semester: 5th
 */

const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 5000;

// ==========================================
// 1. Core Built-in & Third-party Middleware
// ==========================================
app.use(cors());
app.use(express.json());

// ==========================================
// 2. Global Request Logging Middleware
// ==========================================
// Logs method, URL, and ISO timestamp for every incoming request
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});

// ==========================================
// 3. Supplementary Middleware: Content-Type Checker
// ==========================================
// Rejects POST and PUT requests that lack application/json header
const checkContentType = (req, res, next) => {
```

```
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        success: false,
        error: 'Bad Request: Content-Type must be application/json'
      });
    }
  }
  next();
};

// ========================================
// 4. Supplementary Middleware: Task ID Validator
// ========================================
// Validates that :id parameter is a positive integer before hitting controller
const validateTaskId = (req, res, next) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) {
    return res.status(400).json({
      success: false,
      error: `Invalid task ID format '${req.params.id}'. ID must be a positive integer.`
    });
  }
  req.taskId = id;
  next();
};

// ========================================
// 5. In-Memory Data Store (Tasks)
// ========================================
let tasks = [
  { id: 1, title: 'Complete Practical 1 - React Portfolio App', completed: true, createdAt: '2026-10-
18T10:00:00.000Z' },
  { id: 2, title: 'Complete Practical 2 - React Router & State Management', completed: true, createdAt:
'2026-10-19T14:30:00.000Z' },
  { id: 3, title: 'Complete Practical 3 - GitHub REST API Integration', completed: true, createdAt:
'2026-10-21T09:15:00.000Z' },
  { id: 4, title: 'Complete Practical 4 - Express REST API & Middleware', completed: false, createdAt:
'2026-10-21T18:00:00.000Z' }
];

// Server Welcome Route
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Task Manager RESTful API (ITUE301 - Practical 4)',
    student: 'Shubham Pansuriya (24CS057)',
    version: '1.0.0',
    endpoints: {
      getAllTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id',
      simulateError: 'GET /simulate-error'
    }
  });
});

// ========================================
// 6. REST CRUD Endpoints
// ========================================

// GET /tasks -> Retrieve all tasks (Status: 200 OK)
app.get('/tasks', (req, res) => {
  res.status(200).json({
    success: true,
    count: tasks.length,
    data: tasks
```

```
  });
});

// GET /tasks/:id -> Retrieve a single task by ID (Status: 200 OK or 404 Not Found)
app.get('/tasks/:id', validateTaskId, (req, res) => {
  const task = tasks.find((t) => t.id === req.taskId);
  if (!task) {
    return res.status(404).json({
      success: false,
      error: `Task with ID ${req.taskId} not found`
    });
  }
  res.status(200).json({
    success: true,
    data: task
  });
});

// POST /tasks -> Create a new task (Status: 201 Created or 400 Bad Request)
app.post('/tasks', checkContentType, (req, res) => {
  const { title, completed } = req.body;

  if (!title || typeof title !== 'string' || title.trim() === '') {
    return res.status(400).json({
      success: false,
      error: "Task 'title' is required and must be a non-empty string"
    });
  }

  const nextId = tasks.length > 0 ? Math.max(...tasks.map((t) => t.id)) + 1 : 1;
  const newTask = {
    id: nextId,
    title: title.trim(),
    completed: Boolean(completed),
    createdAt: new Date().toISOString()
  };

  tasks.push(newTask);

  res.status(201).json({
    success: true,
    message: 'Task created successfully',
    data: newTask
  });
});

// PUT /tasks/:id -> Update an existing task (Status: 200 OK, 400 Bad Request, or 404 Not Found)
app.put('/tasks/:id', validateTaskId, checkContentType, (req, res) => {
  const task = tasks.find((t) => t.id === req.taskId);
  if (!task) {
    return res.status(404).json({
      success: false,
      error: `Task with ID ${req.taskId} not found`
    });
  }

  const { title, completed } = req.body;

  if (title !== undefined) {
    if (typeof title !== 'string' || title.trim() === '') {
      return res.status(400).json({
        success: false,
        error: "Task 'title' must be a non-empty string"
      });
    }
    task.title = title.trim();
  }

  if (completed !== undefined) {
```

```
    task.completed = Boolean(completed);
  }

  task.updatedAt = new Date().toISOString();

  res.status(200).json({
    success: true,
    message: `Task ${req.taskId} updated successfully`,
    data: task
  });
});

// DELETE /tasks/:id -> Delete a task (Status: 200 OK or 404 Not Found)
app.delete('/tasks/:id', validateTaskId, (req, res) => {
  const index = tasks.findIndex((t) => t.id === req.taskId);
  if (index === -1) {
    return res.status(404).json({
      success: false,
      error: `Task with ID ${req.taskId} not found`
    });
  }

  const deleted = tasks.splice(index, 1)[0];

  res.status(200).json({
    success: true,
    message: `Task ${req.taskId} deleted successfully`,
    deletedTask: deleted
  });
});

// GET /simulate-error -> Deliberate error endpoint to verify global error handler
app.get('/simulate-error', (req, res, next) => {
  try {
    throw new Error('Deliberate internal server error caught by global error middleware pipeline');
  } catch (err) {
    next(err); // Forward error to the global error handling middleware
  }
});

// ========================================
// 7. Supplementary: 404 Handler for Undefined Routes
// ========================================
app.use((req, res, next) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    requestedPath: req.originalUrl,
    method: req.method
  });
});

// ========================================
// 8. Global Error Handling Middleware
// ========================================
// Must have 4 arguments (err, req, res, next) and be defined as the very last middleware
app.use((err, req, res, next) => {
  console.error('[GLOBAL ERROR HANDLER]:', err.message);
  res.status(500).json({
    success: false,
    error: 'Internal Server Error',
    message: err.message || 'Something went wrong on the server'
  });
});

// ========================================
// 9. Start Server
// ========================================
if (require.main === module) {
```

```
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
    console.log(`Ready to accept REST client requests at http://localhost:${PORT}`);
  });
}

module.exports = app;
```

File: package.json (Project Metadata and Dependencies)

```
{
  "name": "task-manager-api-24cs057",
  "version": "1.0.0",
  "description": "Practical 4: Building a RESTful API with Node.js and Express - ITUE301 Advanced Web
Development Frameworks",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": [
    "express",
    "rest-api",
    "crud",
    "middleware",
    "awdf",
    "itue301"
  ],
  "author": "Shubham Pansuriya (24CS057)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.21.2"
  }
}
```

❖ Key Questions & Analysis

Q1: Why must the error handling middleware be defined last in the middleware chain?

In Express, middleware functions execute sequentially in the order they are mounted using app.use(). When a standard route handler encounters an error and calls next(err), Express skips all remaining regular middleware functions and searches downstream for an error-handling middleware (a middleware taking exactly 4 parameters: err, req, res, next). If the error handler is defined before the routes, incoming errors generated inside those routes will bypass the handler, resulting in uncaught exceptions or default HTML error dumps.

Q2: What is the difference between app.use() and a route-specific middleware?

app.use() registers a middleware globally (or for all HTTP verbs under a given base path). Every incoming request matching the path prefix passes through it, making it ideal for cross-cutting concerns like JSON parsing (express.json()), CORS handling, and logging. In contrast, a route-specific middleware is passed as an intermediate argument directly into a specific route definition (e.g., app.get('/tasks/:id', validateTaskId, controller)). It only executes for that exact route and method, making it ideal for localized tasks like parameter validation and route-level authorization.

Q3: Why is it considered bad practice to send raw error stack traces to the client?

Exposing raw stack traces (err.stack) in API responses is a severe security vulnerability. Stack traces reveal internal file system paths, directory structures, third-party package versions, database table names, and server configurations to potential attackers, providing them with critical reconnaissance to craft exploits. In production

environments, APIs should always log the full stack trace server-side for debugging while returning sanitized, user-friendly error messages with generic HTTP 500 status codes to the client.

Q4: What happens if next() is omitted in a custom middleware function?

Express middleware functions must either send an HTTP response (e.g., res.status(400).json(...)) or call next() to pass execution to the subsequent middleware or controller in the pipeline. If a middleware neither sends a response nor invokes next(), the request lifecycle hangs indefinitely until the client or gateway times out.

Q5: Why is app.use(express.json()) required before route definitions?

By default, Node.js streams HTTP request bodies as raw binary chunks. The express.json() built-in middleware intercepts incoming requests with a Content-Type of application/json, buffers the data chunks, parses the raw JSON string into a native JavaScript object, and attaches it to req.body. If routes are declared before express.json(), req.body will remain undefined inside POST and PUT controllers.